

Code-Layout, Readability and Reusability

Date	27 September 2026
Team ID	SWTID-2026-3342
Project Name	AI BlogNest API
Maximum Marks	5 Marks

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	2-space indentation enforced across all .js files
2	Proper File Structure	Files and folders are logically organized	Yes	Server/src/{config, controllers, middleware, models, routes, services} , plus app.js / server.js , and a separate Client/ folder reserved for the frontend
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. connectDB , userSchema , mongoUri
4	Function / Method Names	Functions are descriptively named	Yes	e.g. registerUser , loginUser , generateBlog , summarizeContent
5	Code Comments	Inline and block comments explain logic	Partial	Core modules (config/db.js , controllers) are commented; small helper functions left self-explanatory
6	Modular Design	Code is split into reusable functions/modules	Yes	Routes / Controllers / Middleware / Services / Models decoupled under Server/src
7	No Redundant Code	Duplicate or unused code is removed	Yes	Shared Gemini logic centralized in geminiService.js
8	Error Handling	Exceptions and errors are handled gracefully	Yes	try/catch in connectDB() with process.exit(1) on failure, plus a shared errorMiddleware.js

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Authentication & Role Middleware	Node.js / Express	authRoutes, blogRoutes, aiRoutes - protects every private route and checks role permissions	High
2	geminiService.js	Node.js (Google Gemini AI)	aiController - both the generate-blog and summarize flows	High
3	errorMiddleware.js	Express error-handling middleware	Registered once in app.js, catches errors from every route group	High
4	config/db.js	Mongoose	server.js entry point	High
5	userModel.js / blogModel.js / commentModel.js	Mongoose Schema	authController, blogController, aiController	Medium

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear separation of routes / controllers / middleware / models / services under Server/src
Readability	4	Descriptive names throughout; comments could be denser inside services
Reusability	5	Auth/role middleware and geminiService reused across route groups
Documentation / Comments	4	README.md plus inline comments in core modules (db.js, schemas)
Overall Score	4.5 / 5	Solid, maintainable, extensible codebase ready for the Testing phase